

Part A: Theory

Q1. Difference Between Latency and Throughput

Sr. No.	Latency	Throughput
1	Measures time (usually in milliseconds).	Measures volume (requests per second).
2	Applies to one input from start to finish.	Applies to batch processing or handling many users at once.
3	Critical for real-time applications like voice assistants.	Critical for background servers serving millions of users or processing large data.

Which factors affect model performance the most?

It depends on the application. For real-time applications such as chatbots, **low latency** is more important because users expect quick responses.

For batch processing or large-scale serving, **high throughput** matters more.

Batching can increase throughput, but it may also increase the latency of individual requests.

Q2. KV Cache

KV Cache stands for **Key-Value Cache**. It is used in Transformer-based language models to make text generation faster.

When an LLM generates text, it generates one token at a time. Normally, the model would have to calculate the Key and Value information of previous tokens repeatedly.

KV Cache solves this problem by storing the Key and Value information that has already been calculated. When the model generates the next token, it reuses the stored information instead of recalculating it.

Example:

If the model has already generated:

“I am learning AI”

the Key and Value information for these tokens can be stored in the cache. When generating the next word, the model can reuse them.

This reduces repeated calculations and makes generation faster.

Disadvantage: KV Cache requires extra memory, especially when the sequence becomes very long.

Q3. Quantization

Quantization is a technique used in Large Language Models (LLMs) to convert weights and activation values from high precision data, usually 32-bit floating point (FP32) or 16-bit floating point (FP16), to lower precision formats such as 8-bit integers (INT8).

It works by mapping a continuous range of floating-point values to a discrete set of integers using a scaling factor (S) and a zero-point (Z).

$$Q = \text{round}\left(\frac{r}{S}\right) + Z$$

- **INT8 Quantization:** Offers a good balance. It cuts memory usage by 75% while usually reducing accuracy by less than 1%.
- **INT4 Quantization:** Offers maximum efficiency. It cuts memory usage by 87.5%, but aggressive rounding into only 16 levels may reduce accuracy by 1% to 5% or more.

1. Memory Size Calculation

FP32 Memory:

$$1 \text{ billion parameters} \times 4 \text{ bytes} = 4,000,000,000 \text{ bytes} = 4 \text{ GB}$$

INT8 Memory:

$$1 \text{ billion parameters} \times 1 \text{ byte} = 1,000,000,000 \text{ bytes} = 1 \text{ GB}$$

Memory saved: 3 GB (75%)

INT4 Memory:

$$1 \text{ billion parameters} \times 0.5 \text{ byte} = 500,000,000 \text{ bytes} = 0.5 \text{ GB}$$

Memory saved: 3.5 GB (87.5%)

2. Accuracy Impact Example

Suppose the original FP32 model scores 80% on a reasoning benchmark.

- **INT8 Model:** Accuracy drops slightly to 79.5%.
- **INT4 Model:** Accuracy drops to 75.0% because aggressive rounding may blur subtle patterns in the model weights.

Q4. vLLM and PagedAttention

One popular inference serving framework is **vLLM**.

vLLM is designed for efficiently serving Large Language Models (LLMs). One of its important optimizations is **PagedAttention**.

PagedAttention manages the KV Cache in smaller blocks instead of keeping one large continuous block of memory for every request.

This makes memory usage more efficient and reduces memory wastage.

Because of this, vLLM can handle more requests simultaneously and achieve better throughput when serving LLMs.

Overall, vLLM helps make LLM inference faster and more memory efficient.